

UNIVERSIDAD DEL CAUCA



Visión artificial y control autónomo para el robot industrial UR5e

Autores:

Daniel Alejandro Ruano Bustos
Wilson Antonio Guasca Ipiales

Directores:

Hermes Fabian Vargas Rosero
Martín Alonso Muñoz Medina

**Facultad de ingeniería Electrónica y
Telecomunicaciones
Departamento de Telecomunicaciones
Programa de ingeniería Automatica Industrial
Popayán - Cauca.**

22 de junio de 2026

1. Introducción

La creciente adopción de sistemas robóticos en entornos industriales ha transformado la manera en que se llevan a cabo procesos de manufactura, ensamblaje y manipulación de materiales. Sin embargo, para que los robots puedan desenvolverse de manera eficiente en escenarios dinámicos y cambiantes, ya no es suficiente que ejecuten secuencias de movimientos previamente programadas. La robótica moderna demanda sistemas capaces de percibir su entorno, interpretar información relevante y tomar decisiones que les permitan interactuar con el mundo de forma cada vez más autónoma.

En este contexto, la visión artificial se ha consolidado como una de las tecnologías más importantes dentro del campo de la automatización inteligente. Al dotar a los robots de la capacidad de adquirir y procesar información visual, es posible que estos identifiquen objetos, reconozcan patrones, estimen posiciones y adapten sus acciones a condiciones variables del entorno. Esta capacidad resulta especialmente relevante en aplicaciones de manipulación robótica, donde la localización precisa de piezas y la interacción con elementos de distinta naturaleza constituyen desafíos fundamentales.

El presente trabajo describe el diseño, desarrollo e implementación de un sistema de visión artificial aplicado al robot industrial colaborativo **Universal Robots UR5e**, utilizando como entorno de experimentación la plataforma de simulación **Webots R2025a**. La propuesta desarrollada permite que el robot detecte objetos de colores específicos mediante la cámara instalada en su efector final, procese la información visual en tiempo real y ejecute tareas de agarre y traslado de manera autónoma. Adicionalmente, el sistema incorpora una interfaz web que posibilita la interacción entre el operador humano y el robot mediante comandos escritos en lenguaje natural, facilitando una forma de control intuitiva y accesible.

Desde el punto de vista tecnológico, la solución integra diversas herramientas ampliamente utilizadas en la investigación y el desarrollo de sistemas robóticos. El procesamiento de imágenes se realiza mediante la biblioteca **OpenCV**, encargada de la adquisición, segmentación y detección de objetos en tiempo real. La comunicación entre el simulador y la interfaz de usuario se implementa a través del protocolo **WebSocket**, lo que garantiza un intercambio de información bidireccional eficiente y de baja latencia. Por su parte, el control del manipulador se fundamenta en modelos de cinemática inversa analítica, permitiendo calcular las configuraciones articulares necesarias para alcanzar las posiciones objetivo determinadas por el sistema de visión.

Uno de los aspectos más relevantes de la propuesta es su capacidad de operar de forma completamente *offline*, sin depender de servicios externos de inteligencia artificial ni de plataformas de computación en la nube. Esta característica no solo mejora la velocidad de respuesta del sistema, sino que también incrementa su confiabilidad, privacidad y disponibilidad operativa, aspectos especialmente valorados en aplicaciones industriales donde la continuidad del servicio resulta crítica. Más allá de la implementación específica presentada, este trabajo evidencia cómo la integración de visión artificial, comunicación en tiempo real y técnicas de control robótico puede contribuir al desarrollo de sistemas cada vez más autónomos e inteligentes. Asimismo, constituye una plataforma de experimentación que puede servir como base para futuras investigaciones orientadas a la incorporación de algoritmos de aprendizaje automático, planificación de tareas y colaboración avanzada entre humanos y robots en entornos industriales.

2. Objetivos

2.1. Objetivo General

Desarrollar e implementar un sistema integrado de visión artificial y control por comandos de texto para el robot UR5e en el simulador Webots, capaz de detectar, localizar y manipular objetos de colores en una superficie de trabajo de forma autónoma.

2.2. Objetivos Específicos

1. Configurar el entorno de simulación en Webots R2025a con el robot UR5e, el gripper Robotiq 2F-140 y la cámara de muñeca JetBotRaspberryPiCamera.
2. Implementar un controlador Python que procese las imágenes de la cámara en tiempo real utilizando OpenCV para detectar objetos por segmentación de color en el espacio HSV.
3. Establecer un canal de comunicación bidireccional mediante WebSocket entre el controlador de Webots y una interfaz web de usuario.
4. Diseñar e implementar los algoritmos de cinemática inversa del UR5e para calcular las posiciones articulares necesarias que permitan al gripper alcanzar objetos en posiciones arbitrarias dentro del espacio de trabajo.
5. Desarrollar una interfaz web intuitiva que permita al operador enviar comandos de texto en lenguaje natural y visualizar el feed de la cámara del robot en tiempo real.
6. Validar el sistema completo mediante pruebas de agarre de un cilindro azul y un cubo rojo ubicados sobre una mesa de trabajo.

3. Marco Teórico

3.1. Robot Universal Robots UR5e

El UR5e es un robot colaborativo de 6 grados de libertad (GDL) fabricado por Universal Robots. Sus parámetros cinemáticos según la convención Denavit-Hartenberg son los que se presentan en la Tabla 1.

Cuadro 1: Parámetros Denavit-Hartenberg del UR5e

Joint	Nombre	d_i (m)	a_i (m)	α_i (rad)
1	shoulder_pan_joint	0.1625	0.0000	$\pi/2$
2	shoulder_lift_joint	0.0000	-0.4250	0
3	elbow_joint	0.0000	-0.3922	0
4	wrist_1_joint	0.1333	0.0000	$\pi/2$
5	wrist_2_joint	0.0997	0.0000	$-\pi/2$
6	wrist_3_joint	0.0996	0.0000	0

El alcance máximo del robot es de aproximadamente 0.817 m desde la base, calculado como $a_2 + a_3 = 0,425 + 0,3922 = 0,8172$ m.

3.2. Segmentación de Color en Espacio HSV

El espacio de color **HSV** (Hue, Saturation, Value) es preferido para la detección de objetos por color en visión artificial respecto a RGB, ya que separa la información cromática (tono) de la iluminación (valor), haciendo el sistema más robusto a variaciones de luz.

La segmentación se realiza definiendo rangos de umbral para cada componente:

$$M(x, y) = \begin{cases} 255 & \text{si } H_{min} \leq H \leq H_{max} \wedge S \geq S_{min} \wedge V \geq V_{min} \\ 0 & \text{en otro caso} \end{cases} \quad (1)$$

Para el color rojo se requieren dos rangos debido a que este color se encuentra en los extremos del eje de tono circular ($H \approx 0$ y $H \approx 180$):

$$\text{Rojo}_1 : H \in [0, 8], \quad S \geq 150, \quad V \geq 80 \quad (2)$$

$$\text{Rojo}_2 : H \in [170, 180], \quad S \geq 150, \quad V \geq 80 \quad (3)$$

3.3. Cinemática Inversa

La **cinemática inversa** calcula los ángulos articulares $\theta_1, \dots, \theta_6$ necesarios para que el efector final alcance una posición y orientación deseada en el espacio cartesiano. Para el UR5e con gripper apuntando verticalmente hacia abajo, la solución analítica es:

$$\theta_1 = \text{atan2}(\Delta x, \Delta z) \quad (\text{pan}) \quad (4)$$

$$r = \sqrt{\Delta x^2 + \Delta z^2} \quad (\text{distancia horizontal}) \quad (5)$$

$$h = \Delta y - d_1 - d_6 \quad (\text{altura efectiva}) \quad (6)$$

$$\theta_3 = \arccos\left(\frac{r^2 + h^2 - a_2^2 - a_3^2}{2a_2a_3}\right) \quad (\text{codo}) \quad (7)$$

$$\theta_2 = -(\text{atan2}(h, r) + \text{atan2}(a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3)) \quad (\text{hombro}) \quad (8)$$

$$\theta_4 = -\left(\frac{\pi}{2} + \theta_2 + \theta_3\right) \quad (\text{muñeca 1}) \quad (9)$$

3.4. Comunicación WebSocket

El protocolo **WebSocket** (RFC 6455) establece un canal de comunicación full-duplex sobre TCP. A diferencia de HTTP, no requiere que el cliente inicie cada petición, lo que permite al servidor (controlador Webots) enviar frames de cámara en tiempo real al navegador del operador.

4. Arquitectura del Sistema

El sistema está compuesto por tres capas principales que se comunican entre sí, como se muestra en la Figura 1.

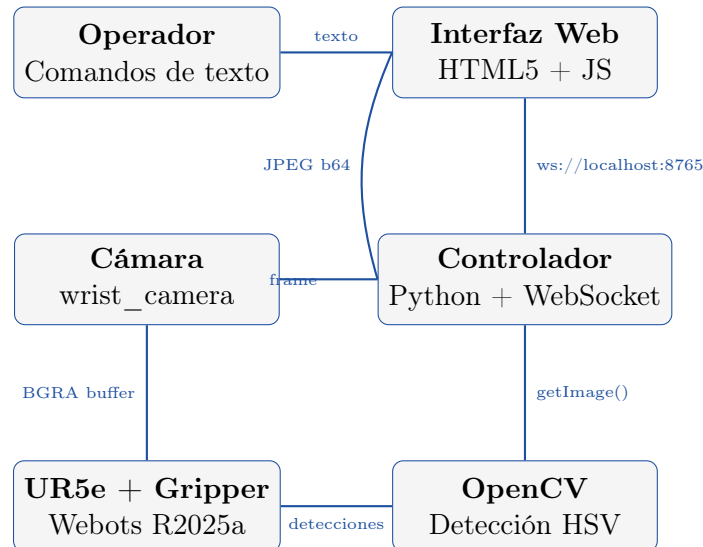


Figura 1: Arquitectura general del sistema de visión artificial

4.1. Descripción de las Capas

Capa de simulación (Webots): Contiene el robot UR5e, el gripper Robotiq 2F-140, la cámara de muñeca y los objetos de la escena. El controlador Python se ejecuta dentro de Webots y tiene acceso directo a todos los dispositivos mediante la API `controller`.

Capa de procesamiento (Python): El controlador realiza el bucle de control principal, captura y procesa imágenes con OpenCV, calcula la cinemática inversa y expone un servidor WebSocket.

Capa de interfaz (Navegador): Una página HTML ejecutada localmente en el navegador muestra el feed de cámara en tiempo real, las detecciones activas y permite enviar comandos de texto al robot.

5. Conexión entre la Interfaz y Webots

5.1. Configuración del Servidor WebSocket

El controlador Python inicia un servidor WebSocket asíncrono en el puerto 8765 al arrancar, usando la librería `websockets` versión 16.x:

```

1 def start_websocket_server():
2     loop = asyncio.new_event_loop()
3     asyncio.set_event_loop(loop)
4 
```

```

5   async def main():
6       async with websockets.serve(handle_client, "localhost", 8765):
7           print("[WS] Escuchando en ws://localhost:8765")
8           await asyncio.Future() # corre indefinidamente
9
10      loop.run_until_complete(main())
11
12  # Ejecutar en hilo separado para no bloquear el bucle de Webots
13  ws_thread = threading.Thread(target=start_websocket_server, daemon=
14      True)
15  ws_thread.start()

```

Listing 1: Inicialización del servidor WebSocket en el controlador

El servidor corre en un hilo demonio separado del bucle principal de Webots, lo que es fundamental: el método `robot.step(TIME_STEP)` debe llamarse continuamente para que la simulación avance, y bloquearlo detendría la física y los sensores.

5.2. Protocolo de Mensajes

La comunicación sigue un protocolo JSON con tres tipos de mensaje:

Cuadro 2: Protocolo de mensajes WebSocket

Dirección	Tipo	Descripción
Web → Robot	command	Comando de texto del operador. El controlador lo interpreta y encola la acción.
Web → Robot	request_frame	Solicitud de imagen. El controlador responde con el último frame JPEG en base64.
Robot → Web	frame	Imagen JPEG codificada en base64, junto con las detecciones activas y el estado del robot.
Robot → Web	decision	Respuesta a un comando: acción ejecutada y mensaje para el operador.

5.3. Polling de Imágenes

La interfaz web solicita un frame cada 500 ms mediante `setInterval`, creando un stream de video a aproximadamente 2 fps:

```

1  setInterval(() => {
2      if (ws && ws.readyState === WebSocket.OPEN) {
3          ws.send(JSON.stringify({ type: "request_frame" }));
4      }
5  }, 500);
6
7  ws.onmessage = (e) => {
8      const data = JSON.parse(e.data);

```

```
9   if (data.type === "frame" && data.data) {  
10       document.getElementById("cam-img").src =  
11           "data:image/jpeg;base64," + data.data;  
12   }  
13 };
```

Listing 2: Polling de frames desde la interfaz web

5.4. Pasos para Establecer la Conexión

Para que el sistema funcione correctamente en Windows con Webots R2025a se deben seguir los siguientes pasos:

1. Instalar Python 3.12 y la librería `websockets`:

```
pip install websockets opencv-python numpy
```

2. Configurar Webots para usar el Python del sistema: **Tools** → **Preferences** → **Python command**: ruta al ejecutable de Python.
3. Colocar el controlador en `controllers/ur5_vision/ur5_vision.py` y asignarlo al nodo del robot en Webots.
4. Ejecutar un servidor HTTP local para la interfaz:

```
python -m http.server 8080
```

5. Abrir `http://localhost:8080/interfaz_ur5.html` en Chrome o Edge.
6. Presionar **Play** en Webots. La interfaz mostrará *WS: conectado* automáticamente.

6. Sistema de Visión Artificial

6.1. Captura de Imagen

La cámara *JetBotRaspberryPiCamera* configurada como **wrist_camera** entrega imágenes de 320×240 píxeles con un campo de visión (*fieldOfView*) de 0.84 radianes. Webots almacena los píxeles en formato **BGRA** (4 canales), que se convierte a BGR para OpenCV:

```
1 def webots_frame_to_bgr(camera):  
2     w = camera.getWidth()    # 320  
3     h = camera.getHeight()   # 240  
4     raw = camera.getImage()   # buffer BGRA  
5     arr = np.frombuffer(raw, dtype=np.uint8).reshape((h, w, 4))  
6     return cv2.cvtColor(arr, cv2.COLOR_BGRA2BGR)
```

Listing 3: Conversión correcta del buffer de cámara de Webots

Nota importante: usar COLOR_RGBA2BGR en lugar de COLOR_BGRA2BGR invierte los canales rojo y azul, produciendo colores incorrectos en la imagen.

6.2. Detección de Objetos por Color

El pipeline de detección para cada objeto sigue cuatro etapas:

1. **Conversión a HSV:** `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`
2. **Umbralización:** `cv2.inRange(hsv, lower, upper)` genera una máscara binaria
3. **Morfología:** apertura morfológica para eliminar ruido pequeño
4. **Contornos:** `cv2.findContours` extrae regiones; se selecciona la de mayor área

```

1  # Rangos HSV para el cilindro azul
2  BLUE_HSV_LOW  = np.array([100, 150, 80])
3  BLUE_HSV_HIGH = np.array([130, 255, 255])
4
5  def detect_blue(frame_bgr):
6      hsv      = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)
7      mask     = cv2.inRange(hsv, BLUE_HSV_LOW, BLUE_HSV_HIGH)
8      kernel   = np.ones((5, 5), np.uint8)
9      mask     = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
10
11     cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
12                               cv2.CHAIN_APPROX_SIMPLE)
13
14     if cnts:
15         c = max(cnts, key=cv2.contourArea)
16         if cv2.contourArea(c) > 350: # filtrar ruido
17             M = cv2.moments(c)
18             if M["m00"] > 0:
19                 cx = int(M["m10"] / M["m00"]) # centroide X
20                 cy = int(M["m01"] / M["m00"]) # centroide Y
21                 return {"x": cx, "y": cy}
22     return None
23
24 \end{lstlisting}
25
26 \subsection{Interpretación de Comandos}
27
28 El sistema no requiere inteligencia artificial externa para
29 interpretar comandos. Se utiliza un clasificador basado en
30 palabras clave que cubre las variantes más comunes del español:
31
32 \begin{lstlisting}[caption={Interpretación de comandos por palabras clave}]
33 def interpret_command(text):
34     t = text.lower().strip()
35     if any(w in t for w in ["suelta", "soltar", "deja", "drop"]):
36         return "place"
37     if any(w in t for w in ["inicio", "home", "reset", "vuelve"]):
38         return "home"
39     if any(w in t for w in ["azul", "blue", "cilindr"]):
40         return "pick_blue"
41     if any(w in t for w in ["rojo", "red", "cubo", "cub"]):
42         return "pick_red"

```



```
39 return "observe"
```

Listing 4: Detección del cilindro azul con OpenCV

6.3. Secuencia de Agarre

Cuando el sistema recibe una orden de agarre, ejecuta la siguiente secuencia:

1. Mover el robot a OBSERVE_POSITION para que la cámara apunte a la mesa
2. Confirmar que el objeto es visible en la imagen
3. Abrir el gripper Robotiq 2F-140
4. Calcular la posición de **pre-agarre** (15 cm sobre el objeto) con IK
5. Mover el brazo al pre-agarre
6. Descender lentamente a la posición de **agarre**
7. Cerrar el gripper
8. Subir a la posición de pre-agarre con el objeto tomado

7. Cinemática Inversa del UR5e

La función de cinemática inversa implementada toma como entrada las coordenadas mundiales del objeto en Webots (t_x, t_y, t_z) y retorna los 6 ángulos articulares necesarios:

```
1 def ik_ur5e(tx, ty, tz):
2     # Desplazamiento desde la base del robot
3     dx = tx - ROBOT_BASE[0]
4     dz = tz - ROBOT_BASE[2]
5
6     # Joint 1: pan (rotacion horizontal)
7     pan = math.atan2(dx, dz)
8
9     # Distancia horizontal al objetivo
10    r = math.sqrt(dx**2 + dz**2)
11
12    # Altura relativa (descontar d1 y longitud del gripper d6)
13    h = ty - ROBOT_BASE[1] - UR5E_D1 - UR5E_D6
14
15    # Verificar alcance
16    reach = math.sqrt(r**2 + h**2)
17    max_reach = UR5E_A2 + UR5E_A3 - 0.01
18
19    if reach > max_reach:
20        scale = max_reach / reach
21        r *= scale; h *= scale; reach = max_reach
22
23    # Joint 3: codo
24    cos_e = (r**2 + h**2 - UR5E_A2**2 - UR5E_A3**2) / (2*UR5E_A2*
25    UR5E_A3)
26    elbow = math.acos(max(-1.0, min(1.0, cos_e)))
```

```

26
27     # Joint 2: hombro
28     alpha = math.atan2(h, r)
29     cos_b = (r**2+h**2+UR5E_A2**2-UR5E_A3**2) / (2*UR5E_A2*reach)
30     beta  = math.acos(max(-1.0, min(1.0, cos_b)))
31     lift  = -(alpha + beta)
32
33     # Joint 4: mu eca para gripper vertical
34     w1 = -(math.pi/2 + lift + elbow)
35
36     return [pan, lift, elbow, w1, -math.pi/2, 0.0]

```

Listing 5: Implementación de cinemática inversa analítica para UR5e

8. Estructura del Proyecto

```

1 mi_proyecto_webots/
2     worlds/
3         escena_ur5.wbt           # Mundo de Webots
4     controllers/
5         ur5_vision/
6             ur5_vision.py       # Controlador principal
7         interfaz_ur5.html       # Interfaz web del operador
8         README.md

```

Listing 6: Estructura de archivos del proyecto

9. Resultados y Discusión

9.1. Detección de Objetos

El sistema logra detectar consistentemente el cilindro azul y el cubo rojo cuando estos se encuentran dentro del campo de visión de la cámara de muñeca. Las áreas de contorno típicas observadas son:

Cuadro 3: Áreas de contorno detectadas en condiciones normales

Objeto	Área mínima (px ²)	Área típica (px ²)
Cilindro azul	350	≈ 2000
Cubo rojo	350	≈ 2400

9.2. Limitaciones Identificadas

- **Falsos positivos en el piso:** Los cuadros del tablero de ajedrez del piso generan detecciones espurias de color rojo. Se mitigó recortando el 30 % inferior de la imagen y elevando el umbral de saturación mínima.

- **Dependencia de posición fija:** La cinemática inversa requiere conocer las coordenadas 3D reales de los objetos, las cuales deben obtenerse desde el árbol de la escena en Webots. Un sistema más robusto requeriría estimación de profundidad mediante una cámara estéreo o sensor ToF.
- **Velocidad de comunicación:** El polling de frames a 500 ms limita la tasa de actualización a 2 fps, suficiente para operación supervisada pero inadecuado para control en lazo cerrado de alta frecuencia.

10. Conclusiones

Se implementó exitosamente un sistema de visión artificial para el robot UR5e en Webots R2025a que integra:

- Detección de objetos en tiempo real por segmentación de color HSV con OpenCV
- Comunicación bidireccional en tiempo real entre el simulador y una interfaz web mediante WebSocket
- Cinemática inversa analítica para el cálculo de trayectorias de agarre
- Interpretación de comandos en lenguaje natural sin dependencia de servicios externos

El sistema demuestra que es posible construir un pipeline completo de visión-acción para un robot industrial simulado utilizando exclusivamente herramientas de código abierto y sin requerir conexión a internet, lo que facilita su uso en entornos educativos y de investigación.

Como trabajo futuro se propone integrar estimación de profundidad para generalizar la detección a posiciones arbitrarias, ampliar el vocabulario de comandos mediante modelos de lenguaje locales (e.g., Llama), e implementar planificación de trayectorias para evitar colisiones.

Referencias

1. Universal Robots. *UR5e Technical Specifications*. Universal Robots A/S, 2023.
2. Cyberbotics Ltd. *Webots Reference Manual R2025a*. <https://cyberbotics.com/doc/reference/>
3. Bradski, G. & Kaehler, A. *Learning OpenCV 4*. O'Reilly Media, 2019.
4. RFC 6455 – *The WebSocket Protocol*. IETF, 2011.
5. Craig, J.J. *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson, 2005.
6. Spong, M.W., Hutchinson, S. & Vidyasagar, M. *Robot Modeling and Control*. Wiley, 2006.